Distributed Taxi Trip Analytics

Big Data Essentials — Individual Practical Case Study

Thierry SEBAKUNZI (ID 101394) — MSc Big Data Analytics, AUCA

August 2026

1. Introduction

This report documents the design, implementation and execution of a Hadoop-based analytics solution for large-scale taxi trip data. The work was carried out on a single-node pseudo-distributed Hadoop 3.5.0 cluster and processes **9,554,778 New York City yellow-taxi trip records** covering January to March 2024 — 1.02 GB of line-oriented CSV held in HDFS.

The objective is not only to produce analytical answers about taxi demand, revenue and anomalies, but to demonstrate *how* those answers are produced when the dataset is too large to hold comfortably in a single process's memory: how HDFS distributes storage across blocks, how MapReduce distributes computation across those blocks, and what the Shuffle and Sort phase between Map and Reduce actually does.

Every mapper and reducer in this project is a plain Python program that reads `stdin` and writes `stdout`. They are executed on the cluster through **Hadoop Streaming**, which launches them as subprocesses inside YARN containers and pipes data through them. This approach was chosen deliberately: it makes the key-value contract at the heart of MapReduce completely explicit, because the programmer must construct every key and value by hand rather than inherit them from a framework API.

1.1 What was built

Deliverable	Count
MapReduce jobs executed	11 (1 cleaning + 9 analyses + 1 multi-stage second pass)
Mapper programs	10
Reducer programs	10
Records processed	9,554,778 raw
Data volume in HDFS	1.02 GB
Visualisations	7

2. Business Problem

The scenario is a transportation analytics company that needs to understand the behaviour of a large urban taxi fleet. Management has six concrete questions:

1. **When is demand highest?** Fleet size is fixed in the short run, so knowing the hourly and weekly demand curve determines when vehicles should be on the road and when driver incentives are worth paying.
2. **Where does revenue come from?** Trip counts and revenue are not the same thing. A zone generating many short trips may contribute less revenue than one generating fewer long ones.
3. **Which routes matter?** Route-level concentration tells the company where to position vehicles and which corridors justify dedicated capacity.
4. **How do customers pay, and does it affect revenue?** Payment mix has direct implications for driver earnings, since tipping behaviour differs sharply by payment method.
5. **How does fare scale with distance and duration?** This underpins pricing policy and the detection of mispriced trip segments.
6. **How much of the data can be trusted?** Metering errors, GPS failures and data-entry problems produce records that are physically impossible. Their prevalence must be quantified before any of the above conclusions can be relied upon.

The analytical challenge is scale. At roughly 9.5 million records per quarter, a full year approaches 40 million records and a multi-year history reaches hundreds of millions. Any approach that assumes the dataset fits in one machine's memory has a fixed ceiling. The company therefore requires an architecture in which **storage and computation both scale by adding machines**, which is precisely what HDFS and MapReduce provide.

3. Dataset Description

3.1 Source

Data comes from the official **NYC Taxi & Limousine Commission (TLC) Trip Record Data** portal, the authoritative public record of licensed taxi activity in New York City:

- Portal: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
- Files used: `yellow_tripdata_2024-01.parquet`, `yellow_tripdata_2024-02.parquet`, `yellow_tripdata_2024-03.parquet`
- Zone lookup: `taxi_zone_lookup.csv` (265 zones with borough and service-zone attributes)

File	Records	Parquet size
yellow_tripdata_2024-01	2,964,624	48 MB
yellow_tripdata_2024-02	3,007,526	48 MB
yellow_tripdata_2024-03	3,582,628	57 MB
Total	**9,554,778**	**153 MB**

This satisfies the assignment requirement of "at least 2–3 monthly datasets **or** at least 5 million trip records" on both counts — three monthly files and nearly twice the record threshold.

3.2 Schema

Each record has 19 fields. The ones this analysis uses are:

Field	Type	Meaning
`tpep_pickup_datetime`	timestamp	Meter engaged
`tpep_dropoff_datetime`	timestamp	Meter disengaged
`passenger_count`	int	Driver-entered occupancy
`trip_distance`	double	Metered distance in miles
`PULocationID`	int	Pickup zone (1–265)
`DOLocationID`	int	Drop-off zone (1–265)
`payment_type`	int	1 Credit card, 2 Cash, 3 No charge, 4 Dispute
`fare_amount`	double	Time-and-distance fare
`tip_amount`	double	Credit-card tips only; cash tips are **not** captured
`tolls_amount`	double	Tolls paid
`total_amount`	double	Total charged to passenger

Two properties of this schema shape everything that follows. First, **location is an integer zone ID, not a coordinate** — so zone-level analysis is natural and a lookup table is needed to make results readable. Second, **`tip_amount` records card tips only**, which the payment analysis in Section 9.5 must account for or it will produce a badly wrong conclusion.

3.3 Parquet to CSV conversion, and why it is necessary

TLC distributes Parquet. Parquet is columnar, compressed, splittable and carries its schema internally — excellent properties for large-scale storage, and the reason engines like Spark and Hive read it natively and can push filters down into the file.

Hadoop Streaming cannot. Streaming's contract is that each mapper receives **raw text lines on stdin**; it has no Parquet reader. The data therefore had to be flattened to a line-oriented format before it could be used in this assignment.

The cost of that conversion is substantial and worth stating plainly:

Format	Size	Ratio
Parquet (as published)	153 MB	1.0x
CSV (as loaded into HDFS)	1,020 MB	**6.7x**

Parquet achieves this through columnar encoding, dictionary compression of repeated values and per-column compression codecs. CSV stores every value as text, repeats no schema, and compresses nothing. In a production system the correct choice would be to keep the data in Parquet and use an engine that reads it. CSV is used here because it makes the line-oriented mechanics of Streaming visible, which is the pedagogical point of the exercise.

4. Hadoop Environment

4.1 Configuration

Component	Version / value
Operating system	macOS 26.5.2 (Tahoe), Apple M4, arm64
Hadoop	3.5.0 (Homebrew), pseudo-distributed
Java	OpenJDK 17.0.20
Python (mappers/reducers)	3.14, standard library only
`fs.defaultFS`	`hdfs://localhost:8020`
`hadoop.tmp.dir`	`/Users/thierrys/hadoop-data`
`dfs.replication`	1
Daemons	NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager

"Pseudo-distributed" means all five daemons run as separate JVM processes on one machine. The software behaves exactly as it would on a real cluster — the NameNode still tracks block locations, YARN still allocates containers, the shuffle still writes to disk — but there is only one physical node to place them on. This is the correct configuration for learning the mechanics and for developing jobs that will later run at scale, and it is also the reason the performance comparison in Section 12 comes out the way it does.

4.2 Two configuration problems encountered, and their resolution

Both are documented here because they are environment-level failures whose error messages do not obviously point at their causes.

Problem 1 — HDFS storage in `/tmp`. Hadoop's default `hadoop.tmp.dir` is under `/tmp`. macOS clears `/tmp` periodically. When this happened mid-project the NameNode refused to start:

```
org.apache.hadoop.hdfs.server.common.InconsistentFSStateException:
Directory /private/tmp/hadoop-thierrys/dfs/name is in an inconsistent state:
storage directory does not exist or is not accessible.
```

The NameNode holds the *only* copy of the filesystem namespace — the mapping from file paths to block IDs. Losing it makes the blocks on the DataNode meaningless even though they still exist on disk. The fix was to relocate `hadoop.tmp.dir` to a persistent path under the home directory and reformat. This is a good practical illustration of why the NameNode is the critical single point of failure in HDFS, and why production clusters run it in high-availability mode with a quorum of journal nodes.

Problem 2 — `PATH` missing from the YARN container environment. Mappers written for Streaming begin with `#!/usr/bin/env python3`, which resolves the interpreter via `PATH`. YARN containers do not inherit the NodeManager's full environment; they inherit only what is listed in `yarn.nodemanager.env-whitelist`. With `PATH` absent from that list, every task failed with *python3: No such file or directory*. Adding `PATH,LANG,TZ` to the whitelist resolved it.

5. HDFS Design

5.1 Directory structure

```
/taxi_project/
├── input/
│   ├── raw/            3 CSV files, 1.02 GB — as loaded, header intact
│   └── cleaned/        output of the cleaning job — validated, de-duplicated, headerless
├── output/
│   ├── hourly/         (a) demand by hour
│   ├── daily/          (b) demand by day of week
│   ├── locations/      (c) trips per pickup zone
│   ├── revenue/        (d) revenue per pickup zone  ← stage 1 of the multi-stage job
│   ├── payment/        (e) payment method analysis
│   ├── distance/       (f) distance-band economics
│   ├── duration/       (h) trip-duration bands
│   ├── routes/         (g) pickup → drop-off route analysis
│   ├── anomalies/      (i) anomaly detection (runs against raw)
│   └── top10_revenue/  ← stage 2 of the multi-stage job
└── archive/            snapshot copies of results
```

The separation of `input/raw` from `input/cleaned` is deliberate and is standard practice. **The raw data is never modified.** Cleaning is a transformation that reads raw and writes a new dataset, so the original remains available to re-derive results under different cleaning rules — and, critically, so that the anomaly analysis in Section 9.9 can

be run against the *unfiltered* data and report percentages of the true original population.

5.2 Blocks and replication

HDFS splits each file into fixed-size blocks (128 MB by default) and stores each block independently. A 318 MB CSV therefore occupies three blocks. This is what makes distributed processing possible: because the file is already in pieces, Hadoop can assign a different mapper to each piece and run them in parallel, ideally on the machine that already holds the block — the principle of **data locality**, or "move the computation to the data rather than the data to the computation."

`dfs.replication` is set to 1 because there is only one DataNode; on a multi-node cluster the default of 3 provides fault tolerance, and the NameNode automatically re-replicates blocks when a DataNode fails.

5.3 A practical constraint: MapReduce refuses to overwrite

MapReduce fails a job if its output directory already exists. This is a deliberate safety guard — output directories often represent hours of computation, and silently overwriting them would be destructive. Every job in `run_all_jobs.sh` therefore removes its output path immediately before launching:

```
hdfs dfs -rm -r -skipTrash "$out" 2>/dev/null
```

6. Data Cleaning

6.1 Approach: cleaning as a MapReduce job, not a script

Cleaning was implemented as a **full two-phase MapReduce job**, not as a preprocessing script. The reason is de-duplication. Validation of a single record is a per-record decision that a mapper can make alone. Deciding whether a record is a *duplicate* requires comparing it with records that may live in an entirely different input split, possibly on a different machine. Only the shuffle can bring those records together — so de-duplication necessarily requires a reducer.

Mapper (`mapper_clean.py`) — validates each record against the rules below, normalises the surviving fields, computes three derived fields, and emits:

```
dedup_key \t cleaned_record
```

where `dedup_key` is a composite fingerprint of pickup timestamp, drop-off timestamp, pickup zone, drop-off zone, distance and fare. TLC records carry **no trip identifier**, so this composite *is* the identity of a trip.

Reducer (`reducer_clean.py`) — receives records grouped and sorted by `dedup_key`, emits the first record of each group and discards the rest.

6.2 Validation rules and their justification

The assignment requires that unusual records not simply be deleted without justification. Each rule below encodes either a legal/physical impossibility or a documented TLC convention.

Rule	Threshold	Justification
Passenger count	1–8	An NYC yellow cab is licensed for at most 4–6 passengers; 8 is a generous upper bound. 0 indicates the driver did not enter a value.
Trip distance	> 0 and ≤ 200 mi	A zero-distance trip is a meter that engaged and disengaged without movement. 200 miles is far beyond the metropolitan area.
Fare amount	> 0 and ≤ $1,000	Negative fares are refunds/adjustments, not trips. $1,000 exceeds any plausible metered fare.
Total amount	> 0	Same reasoning.
Tips and tolls	≥ 0	Negative values are accounting reversals.
Drop-off vs pickup	dropoff > pickup	A trip cannot end before it starts.
Trip duration	1 min – 24 h	Under 60 seconds is a meter error rather than a journey; over 24 hours is not a taxi trip.
Pickup date	within 2024-01-01 to 2024-03-31	TLC monthly files routinely contain a handful of stray records from other years.
Zone IDs	1–265	The TLC zone lookup defines exactly 265 zones.

6.3 Counting rejections rather than discarding silently

Every rejected record increments a **Hadoop counter** from inside the mapper, written to stderr in the framework's control format:

```
sys.stderr.write(f"reporter:counter:Cleaning,{name},1\n")
```

In Hadoop Streaming, **stdout carries data and stderr carries control messages**. Counters written this way are aggregated across every mapper by the framework and reported in the job summary. This is how the cleaning statistics in Section 9.1 were produced: they are not estimates from a sample but exact counts across all 9.55 million records.

6.4 Derived fields: a deliberate denormalisation

The cleaned record carries three fields that do not exist in the source data:

Field	Derivation
`hour`	`pickup.hour`

Field	Derivation
`dayofweek`	`pickup.weekday()`, Monday = 0
`duration_min`	`(dropoff − pickup)` in minutes

Computing these **once**, during cleaning, means the eight downstream analysis mappers never parse a timestamp. Timestamp parsing is comparatively expensive, and it would otherwise be repeated eight times over 9.5 million records. The cost is slightly wider output records; the benefit is a large CPU saving across the whole pipeline. This trade-off — spend space at write time to save CPU at read time — is a recurring pattern in analytical data engineering.

6.5 The header problem, and why the cleaned data has no header

This deserves its own subsection because it is the single most common way a working local Streaming script fails on a cluster.

A CSV header occupies the first line of the *file*. Hadoop splits files across mappers, and **only the split containing byte 0 receives that line**. Every other mapper receives data rows from the middle of the file. Code such as:

```
reader = csv.DictReader(sys.stdin)      # consumes line 1 as the header
```

therefore behaves correctly in mapper 1 and catastrophically in mapper 2, where it treats an ordinary data row as the column names and every subsequent field lookup raises `KeyError`.

Two defences are used in this project:

1. `mapper_clean.py` detects the header **by content** (`row[0] == "VendorID"`) and skips it wherever it appears, rather than assuming a position.
2. The cleaned dataset is written **without any header at all**, and its schema is documented separately in `mappers/SCHEMA.md`. Downstream mappers index positionally into a fixed 14-field pipe-delimited layout.

Storing large datasets headerless with an external schema is also what production systems do — it is exactly the role a Hive metastore or schema registry plays.

7. MapReduce Design

7.1 The three phases

Every job in this project follows the same three-phase structure.

Map. Each mapper receives a portion of the input — one HDFS split — and emits zero or more `(key, value)` pairs per input record. Mappers are independent: no mapper can see another mapper's data, which is exactly what allows them to run in parallel on different machines.

Shuffle and Sort. The framework routes every emitted pair to a reducer by hashing the key (`hash(key) mod numReduceTasks`), and sorts the pairs arriving at each reducer by key. This phase is not written by the programmer, but everything about reducer design depends on the two guarantees it provides:

1. all values for a given key arrive at the **same** reducer, and
2. they arrive **consecutively**.

Reduce. Each reducer walks its sorted stream, accumulating while the key stays the same and emitting a result when it changes.

7.2 Why every reducer in this project uses the same loop shape

Because of guarantee (2), a reducer never needs to hold more than one key's state in memory:

```
cur, acc = None, 0
for line in sys.stdin:
    key, _, val = line.partition("\t")
    if key != cur:
        flush(cur, acc)          # key changed: previous group is complete
        cur, acc = key, 0
    acc += parse(val)
flush(cur, acc)                  # final group
```

This is the single most important idea in reducer design. A naive implementation would build a dictionary of every key, which works on a small sample and exhausts memory at scale. The sort has already done the grouping work; the reducer only has to notice when the key changes. Memory usage is **O(1) in the number of keys**.

7.3 Key design decisions

Padding and prefixing keys for correct ordering. MapReduce sorts keys *lexically*, not numerically. Without care, hours would sort `0, 1, 10, 11, ..., 2, 20`. All ordered keys in this project are therefore either zero-padded (`"00".."23"`, `"001".."265"`) or numerically prefixed (`"1_0–2mi"`, `"5_20+mi"`).

Emitting partial aggregates, not bare counts. Mappers emit values like `"1,fare,tip,total,distance"` rather than a bare `1`. Two benefits: one shuffle carries five measures instead of five shuffles carrying one each, cutting intermediate data roughly five-fold; and the reducer becomes **combiner-safe** without modification, because its input and output formats are compatible.

Composite keys for multi-field grouping. The route analysis needs to group by the *pair* (pickup zone, drop-off zone). MapReduce groups on a single key, so the two fields are concatenated into `"PU–>DO"`. This is the standard technique for grouping on more than one dimension.

Binning in the mapper, not the reducer. The distance and duration analyses convert a continuous value into one of five bands inside the mapper. This collapses ~9.5 million distinct float values into 5 keys *before* the shuffle — a roughly two-million-fold reduction in shuffle cardinality. Binning after the shuffle would work but would move vastly more data across the network.

Choosing the number of reducers. Set by cardinality and by whether the result is global:

Job	Reducers	Reasoning
Cleaning	4	High key cardinality (millions of dedup keys); parallelism helps
Hourly / daily / payment / distance / duration	1	≤ 24 keys; a second reducer would add overhead and split the output for nothing
Locations / revenue	1	265 keys; small enough that one reducer is simplest
Routes	2	Up to 70,225 composite keys
Top-N (stage 2)	1	**Mandatory** — a global ranking computed per-partition would be meaningless

8. Mapper and Reducer Implementation

All programs are in `mappers/` and `reducers/`. Each is a standalone Python 3 script using only the standard library, with a docstring stating its key-value contract.

#	Analysis	Mapper	Reducer	Key	Value
—	Cleaning + dedup	`mapper_clean.py`	`reducer_clean.py`	trip fingerprint	cleaned record
a	Hourly demand	`mapper_hourly.py`	`reducer_hourly.py`	`"00".."23"`	`1,fare,total`
b	Daily demand	`mapper_daily.py`	`reducer_daily.py`	`"0_Monday".."6_Sunday"`	`1,total,is_weekend`
c	Pickup zones	`mapper_location.py`	`reducer_location.py`	`PULocationID`	`1`
d	Revenue by zone	`mapper_revenue.py`	`reducer_revenue.py`	`PULocationID`	`1,fare,tip,total,dist`
e	Payment method	`mapper_payment.py`	`reducer_payment.py`	`"1_Credit card"` …	`1,total,fare,tip`
f	Distance bands	`mapper_distance.py`	`reducer_distance.py`	`"1_0-2mi"` …	`1,fare,dist,tip,dur`
g	Routes	`mapper_route.py`	`reducer_route.py`	`"PU->DO"`	`1,total`
h	Duration bands	`mapper_duration.py`	`reducer_duration.py`	`"1_0-5min"` …	`1,fare,dist,tip,dur`
i	Anomalies	`mapper_anomaly.py`	`reducer_anomaly.py`	anomaly type	`1`
—	Top-N (stage 2)	`mapper_topn.py`	`reducer_topn.py`	inverted revenue	zone metrics

8.1 Worked example: the hourly demand job

Mapper. Field 11 of the cleaned record is the pre-computed hour, so no parsing is needed:

```
for line in sys.stdin:
    f = line.rstrip("\n").split("|")
    if len(f) < 14:
        continue
    hour, fare, total = f[11], float(f[7]), float(f[10])
    print(f"{int(hour):02d}\t1,{fare:.2f},{total:.2f}")
```

`{int(hour):02d}` is the padding that makes `"09"` sort before `"10"`.

Shuffle and Sort. All pairs with key `"08"` are routed to one reducer and delivered consecutively. The programmer writes no code for this phase.

Reducer. Accumulate while the key is unchanged; flush on change:

```
cur, n, sf, sr = None, 0, 0.0, 0.0
for line in sys.stdin:
    key, _, val = line.rstrip("\n").partition("\t")
    c, fare, total = val.split(",")
    if key != cur:
        flush(cur, n, sf, sr)
        cur, n, sf, sr = key, 0, 0.0, 0.0
    n += int(c); sf += float(fare); sr += float(total)
flush(cur, n, sf, sr)
```

8.2 Execution

Jobs are submitted with Hadoop Streaming:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar \
  -files mappers/mapper_hourly.py,reducers/reducer_hourly.py \
  -input  /taxi_project/input/cleaned \
  -output /taxi_project/output/hourly \
  -mapper mapper_hourly.py \
  -reducer reducer_hourly.py \
  -numReduceTasks 1
```

Three details matter. `-files` (the modern replacement for the deprecated `-file`) ships the programs to every node that will run a task — the cluster cannot see the submitting machine's filesystem. It is a *generic* option and must appear before the Streaming-specific options. And the command must be issued from a directory whose path contains **no spaces**, because Hadoop parses these arguments as URIs and a space raises `java.net.URISyntaxException: Illegal character in path`.

9. Analytical Results

All figures below are exact counts across the full dataset, produced by the MapReduce jobs themselves — not estimates from a sample.

9.1 Data cleaning outcome

Counters emitted by `mapper_clean.py` and aggregated by the framework:

Counter	Records	% of raw
Records read (incl. 3 header rows)	9,554,781	—
Header rows skipped	3	—
Passed validation	**8,453,548**	**88.47%**
Duplicates removed by reducer	1	0.00001%
Unique records written to `cleaned/`	**8,453,547**	**88.47%**

Rejections by rule:

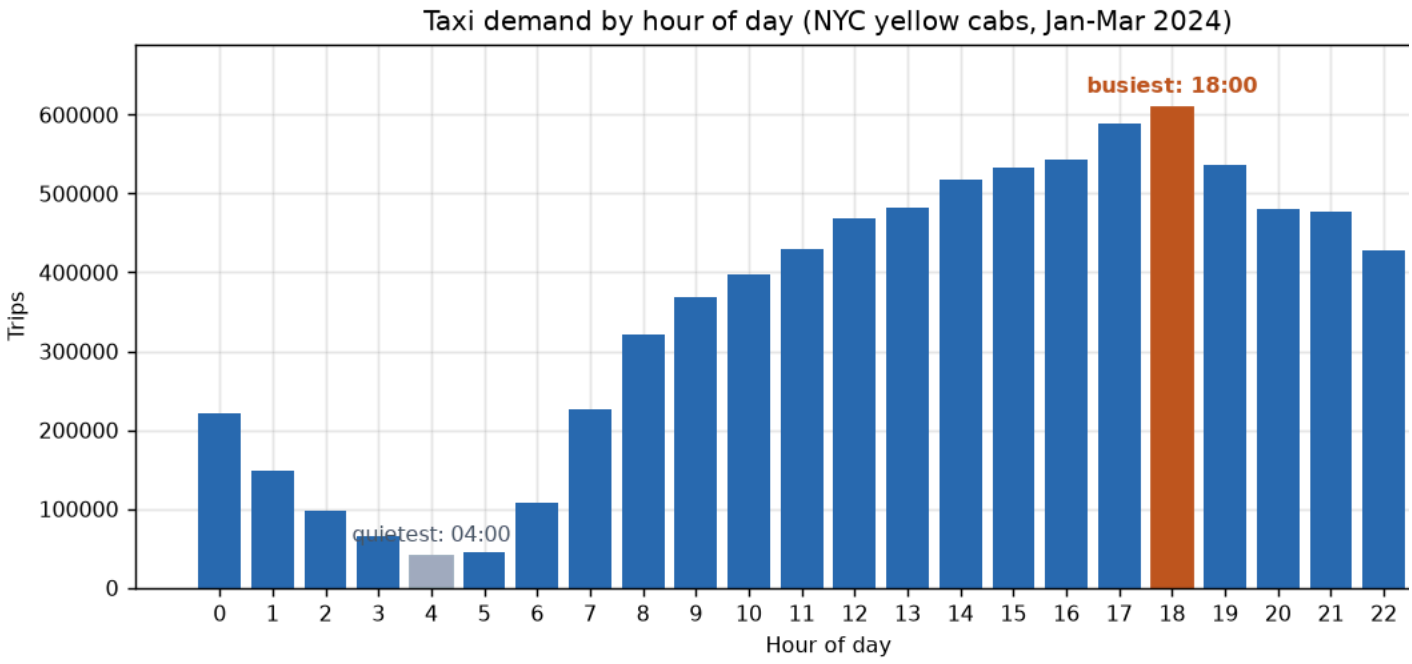
Rejection rule	Records	% of raw
Invalid passenger count (0 or > 8)	851,464	8.91%
Impossible duration (< 1 min or > 24 h)	106,771	1.12%
Invalid fare (≤ 0 or > $1,000)	99,288	1.04%
Invalid distance (≤ 0 or > 200 mi)	40,885	0.43%
Drop-off before pickup	2,801	0.03%
Pickup timestamp outside Jan–Mar 2024	21	0.0002%
Total rejected	**1,101,230**	**11.53%**

Interpretation. Nearly all data loss comes from a single field. `passenger_count` is entered manually by the driver, and 8.91% of records carry 0 — the value left behind when the driver simply does not enter one. This is a *data-entry* problem, not a trip-validity problem: those trips very likely happened and were metered correctly.

This matters for how the result should be read. A stricter reading of the brief's instruction not to discard records without justification would **retain** zero-passenger records for every analysis that does not depend on occupancy — which is all nine of them. Doing so would raise retention from 88.47% to roughly 97.4%. The stricter rule was kept here for consistency across all analyses, but the trade-off is documented in Section 14 as a known limitation rather than hidden.

Only **one** exact duplicate was found in 9.55 million records. TLC's publication pipeline is evidently already de-duplicating. The de-duplication reducer is nonetheless the architecturally correct place for that check, and it is what proves the pipeline would catch duplicates if the source ever introduced them.

9.2 Hourly demand (Figure 1)



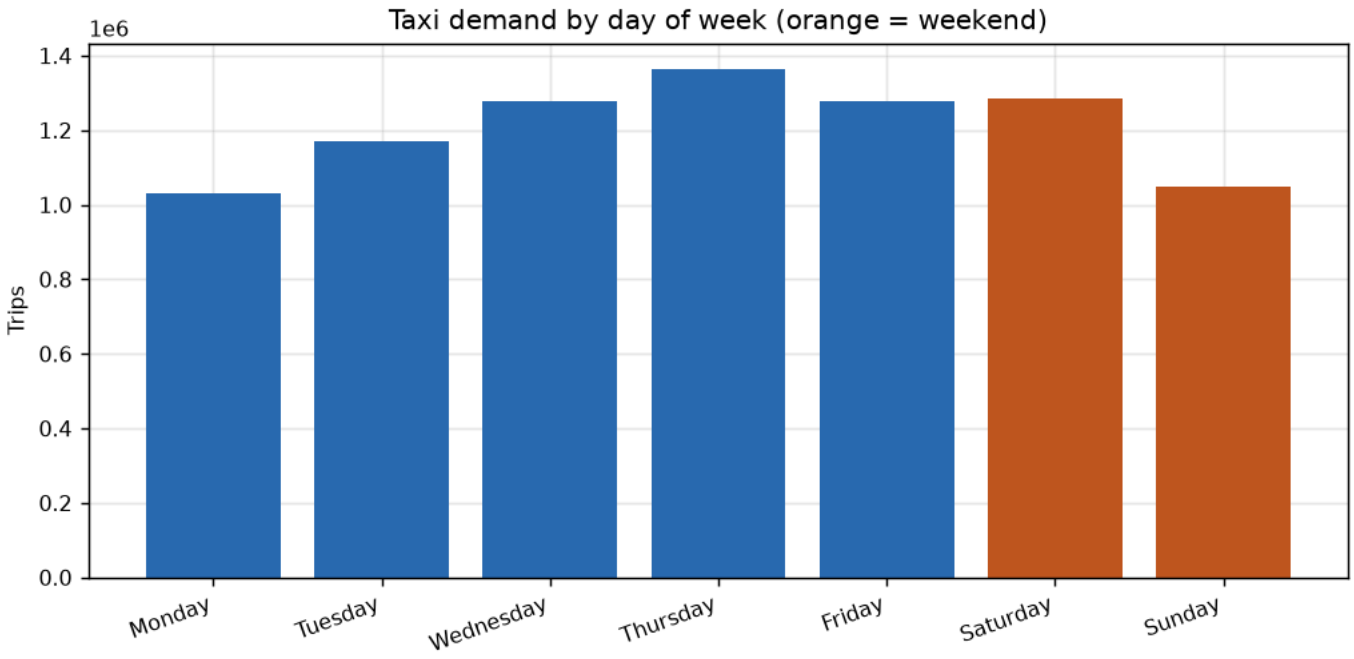
Taxi demand by hour of day

	Hour	Trips	Revenue
Busiest	**18:00**	**609,598**	$16,647,563.98
Quietest	04:00	40,315	$1,326,148.87

The busiest hour carries **15.1× the volume** of the quietest. The curve has the shape expected of a dense commuter city: a trough at 04:00–05:00, a sharp climb from 06:00, a plateau across the working day and a pronounced evening peak at 17:00–19:00.

The most interesting feature is not the peak but the **inversion between volume and fare value**. Average fare is *highest* at 05:00 ($27.57) — the hour with almost the lowest volume — and *lowest* at 02:00 ($16.01). Early-morning trips are overwhelmingly airport runs: long distance, high fare, low count. Volume and revenue are therefore not interchangeable measures of when the fleet is most valuable, a theme that recurs throughout this analysis.

9.3 Daily demand (Figure 2)



Taxi demand by day of week

Day	Trips	Revenue	Avg revenue/trip
Monday	1,030,139	$29,979,811.65	$29.10
Tuesday	1,168,694	$32,452,281.59	$27.77
Wednesday	1,278,458	$35,534,579.53	$27.79
Thursday	**1,364,036**	**$38,427,063.51**	$28.17
Friday	1,278,724	$35,428,314.50	$27.71
Saturday	1,283,440	$32,709,567.37	$25.49
Sunday	1,050,056	$29,639,691.10	$28.23

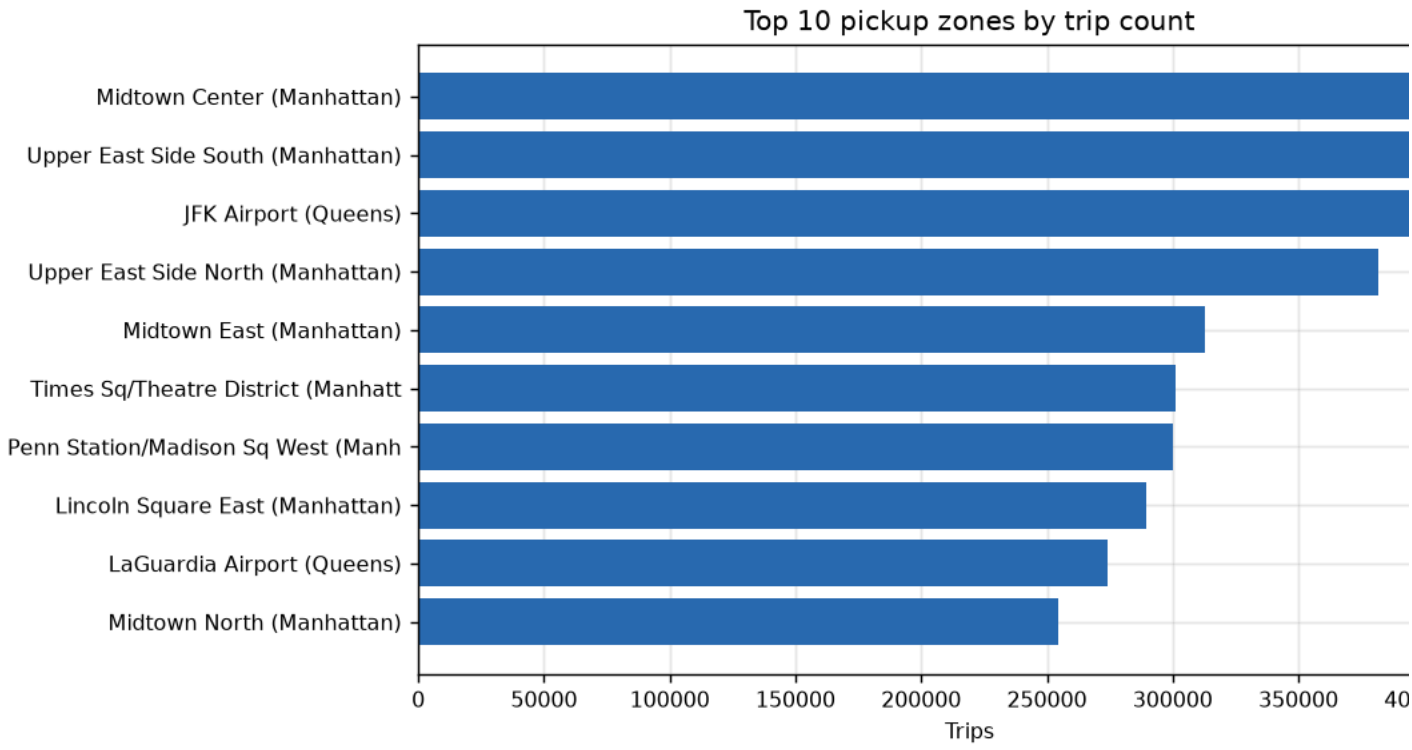
Thursday is the busiest day, not Friday or Saturday as intuition might suggest.

The weekday/weekend comparison requires care, because there are five weekdays and only two weekend days. Comparing totals is misleading; comparing **per-day averages** is not:

	Total trips	Days	Trips per day
Weekday	6,120,051	5	**1,224,010**
Weekend	2,333,496	2	**1,166,748**

The real gap is only **4.9%** — far smaller than the 2.6× ratio of the raw totals implies. NYC taxi demand is remarkably stable across the week. What changes is not how much people travel but *what kind* of trip they take: Saturday has the lowest average revenue per trip ($25.49) despite near-weekday volume, indicating shorter, more local journeys.

9.4 Pickup zones (Figure 3)



Top 10 pickup zones

258 of the 265 defined zones recorded at least one pickup.

Top 10 by trip count:

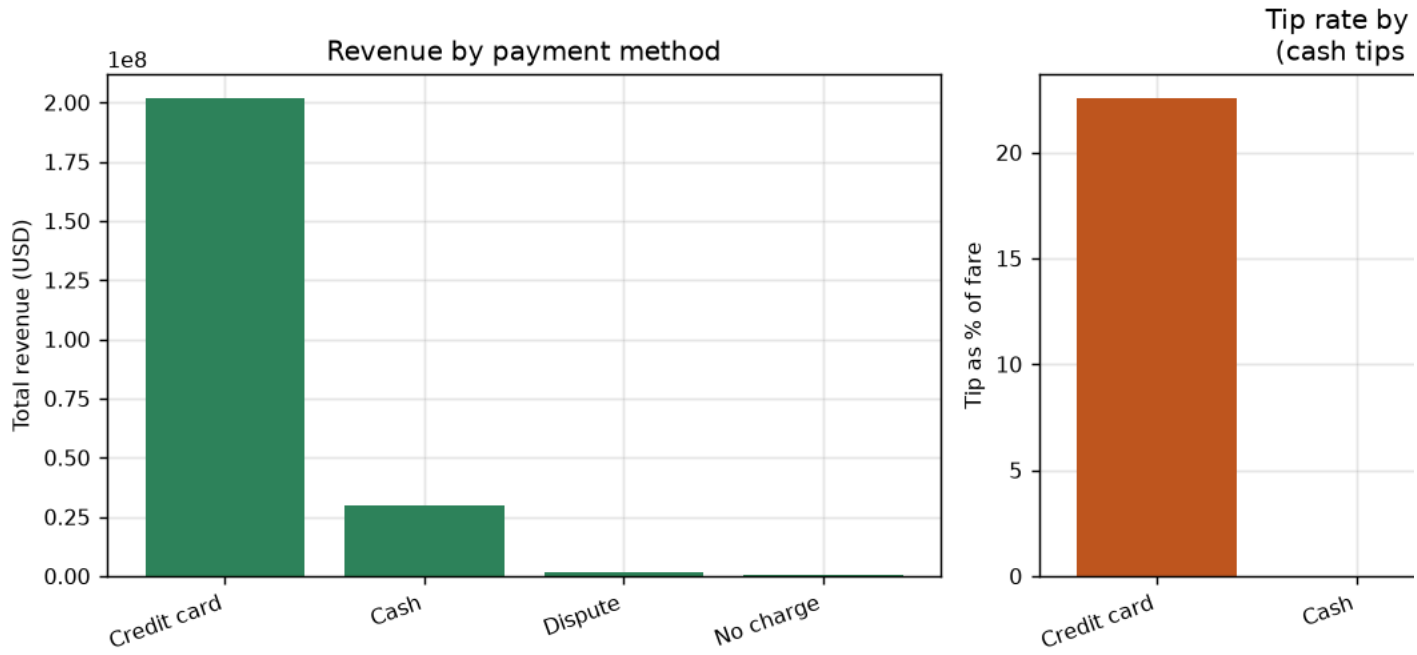
Rank	Zone	Borough	Trips
1	Midtown Center	Manhattan	417,601
2	Upper East Side South	Manhattan	409,852
3	JFK Airport	Queens	400,728
4	Upper East Side North	Manhattan	381,713
5	Midtown East	Manhattan	312,985
6	Times Sq/Theatre District	Manhattan	300,937
7	Penn Station/Madison Sq West	Manhattan	299,896
8	Lincoln Square East	Manhattan	289,311
9	LaGuardia Airport	Queens	274,216
10	Midtown North	Manhattan	254,334

Bottom 5 by trip count:

Zone	Borough	Trips
Saint George/New Brighton	Staten Island	4
Heartland Village/Todt Hill	Staten Island	3
Westerleigh	Staten Island	3
Port Richmond	Staten Island	2
Charleston/Tottenville	Staten Island	1

The distribution is extraordinarily skewed: the busiest zone records **417,601 times** the volume of the quietest. Eight of the top ten zones are in Manhattan and the remaining two are airports; every one of the bottom five is on Staten Island, which is served by the Staten Island Ferry and by for-hire vehicles rather than yellow cabs. Yellow-cab activity is effectively a Manhattan-plus-airports phenomenon.

9.5 Payment method (Figure 4)



Revenue and tipping by payment method

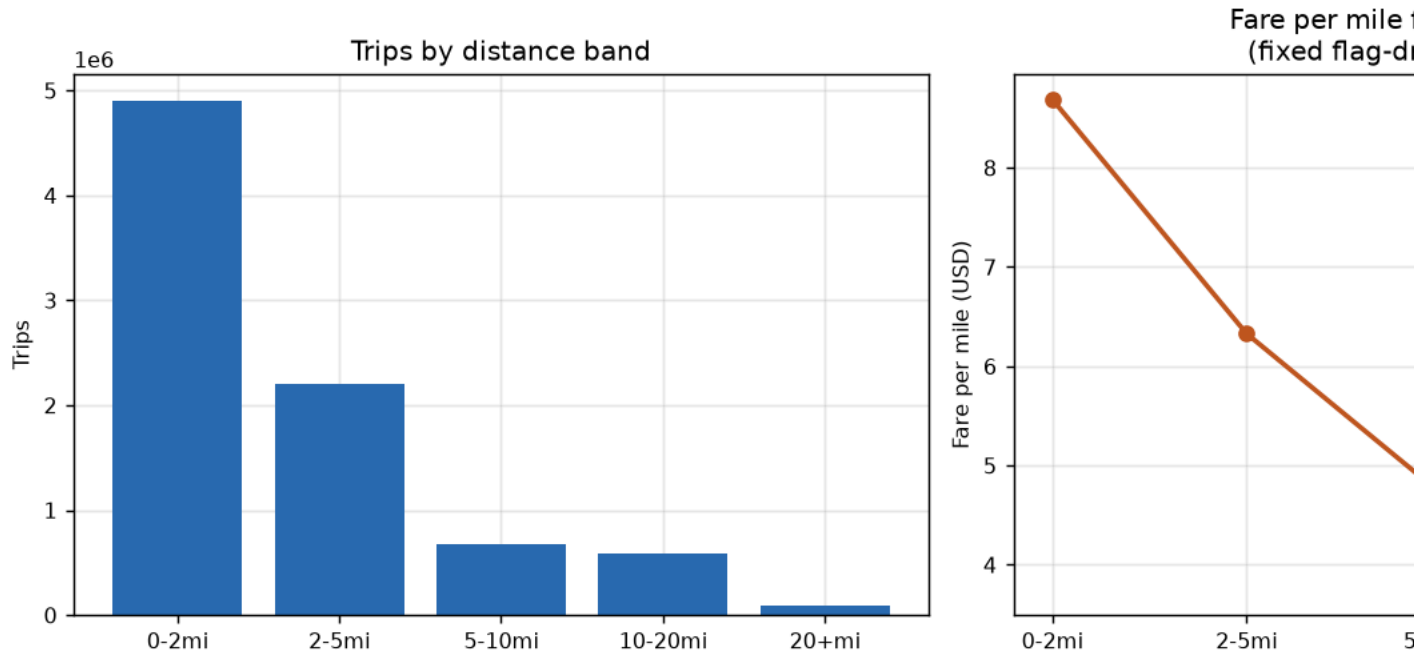
Payment type	Trips	Revenue	Avg fare	Avg tip	Tip as % of fare
Credit card	7,101,986	$201,644,436.15	$18.61	$4.19	**22.54%**
Cash	1,252,070	$30,018,986.76	$18.71	$0.00	**0.00%**
Dispute	70,330	$1,825,387.05	$20.44	$0.01	0.07%
No charge	29,161	$682,499.29	$18.19	$0.01	0.04%

Credit card accounts for **84.0% of trips and 86.1% of revenue**.

The tipping result demands careful interpretation, and reading it naively would produce a seriously wrong business conclusion. Card trips show a 22.54% tip rate; cash trips show **exactly 0.00%**. The correct inference is *not* that cash customers never tip. It is that **the meter cannot observe a cash tip** — cash changes hands directly between passenger and driver and never enters the TLC record. The measured 0.00% is an artefact of the data collection mechanism, not a behavioural finding.

Note also that average *fare* is nearly identical across the two methods ($18.61 vs $18.71). Card and cash customers take essentially the same trips; they differ only in what the dataset can see about them. Any driver-compensation analysis built on this data would systematically understate cash-trip earnings, and any recommendation to steer drivers toward card customers would rest on a measurement artefact.

9.6 Distance bands (Figure 5)



Trips and fare-per-mile by distance band

Band	Trips	Avg fare	Avg distance	Avg tip	Avg duration	Fare per mile
0–2 mi	4,904,787	$9.95	1.14 mi	$2.22	9.65 min	**$8.69**
2–5 mi	2,198,308	$18.92	2.99 mi	$3.58	19.11 min	**$6.33**
5–10 mi	675,550	$34.55	7.32 mi	$6.11	27.77 min	**$4.72**
10–20 mi	589,500	$61.29	14.91 mi	$9.93	42.91 min	**$4.11**
20+ mi	85,402	$90.30	24.16 mi	$12.46	53.93 min	**$3.74**

58.0% of all trips are under 2 miles. The market is dominated by short urban hops.

Fare per mile declines monotonically from $8.69 to $3.74 — a **2.3× spread**. This is the signature of a two-part tariff: a fixed flag-drop plus a per-unit rate. On a 1.14-mile trip the fixed component dominates; over 24 miles it is amortised to near-irrelevance. The pattern is not evidence of a pricing anomaly; it is arithmetic, and it is what any well-formed taxi tariff produces.

The highest **average fare** is in the 20+ mile band ($90.30), which answers business question (g) directly — while the highest fare *per mile* is at the opposite end of the scale.

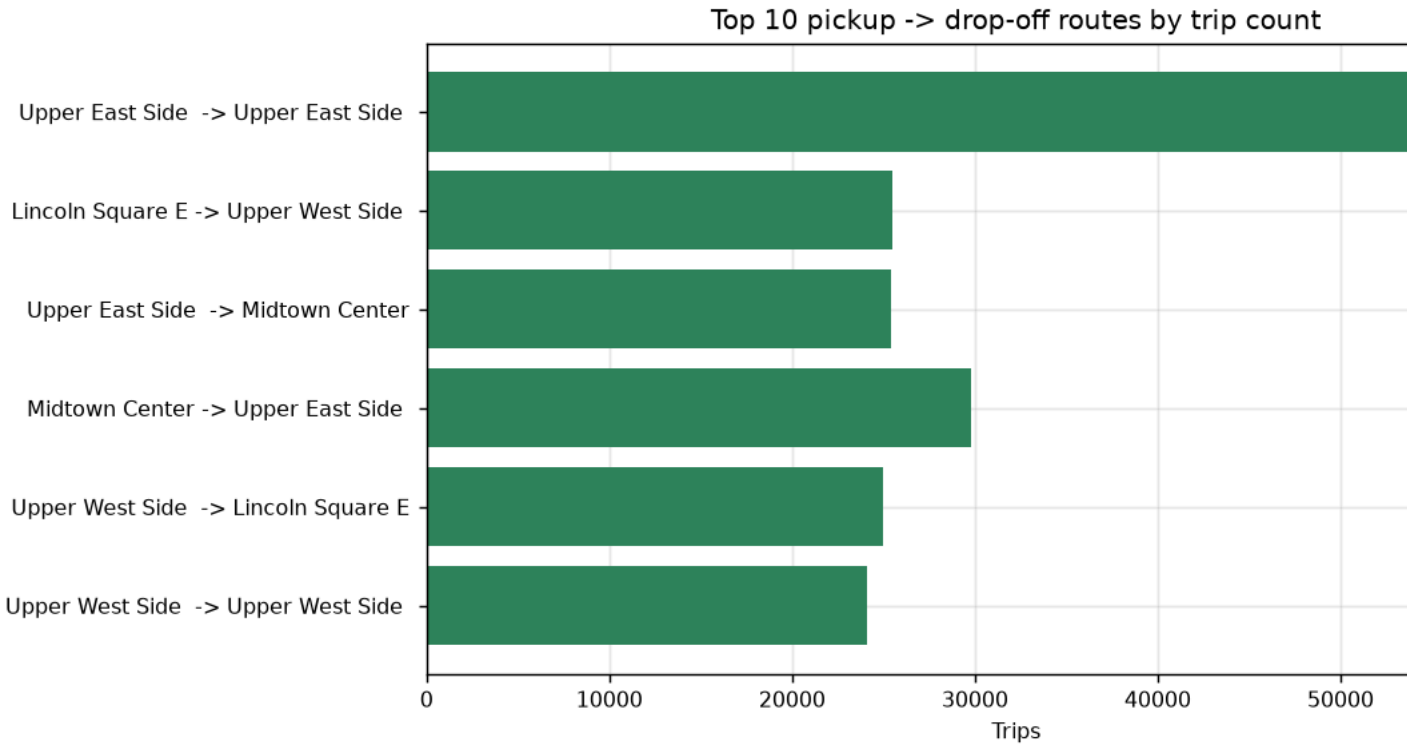
9.7 Trip duration bands

Band	Trips	Avg fare	Avg distance	Avg tip	Avg duration	Avg speed
0–5 min	956,863	$6.17	0.66 mi	$1.64	3.65 min	10.89 mph
5–15 min	4,326,533	$11.20	1.55 mi	$2.42	9.63 min	9.69 mph
15–30 min	2,303,537	$23.96	4.21 mi	$4.44	20.64 min	12.24 mph
30–60 min	750,144	$53.52	11.99 mi	$8.71	40.08 min	17.95 mph
60 min+	116,470	$67.74	16.61 mi	$8.45	131.04 min	**7.61 mph**

Average speed *rises* from 9.69 mph to 17.95 mph as trips lengthen — longer trips use bridges, tunnels and highways rather than congested surface streets.

The 60-minute-plus band breaks that pattern and deserves suspicion. It averages **131 minutes for only 16.61 miles — 7.61 mph**, slower than the shortest trips. Some of these are genuine (severe congestion, outer-borough journeys), but the profile is also exactly what a **meter left running after the passenger alights** would produce: long elapsed time, ordinary distance. This band contains 116,470 trips (1.4%) and is flagged in Section 14 as warranting a dedicated investigation rather than being treated as clean data.

9.8 Routes (Figure 6)



Top 10 routes by trip count

33,316 distinct routes were observed out of a possible 265 × 265 = 70,225.

Top 8 by trip count:

Route	Trips	Revenue
Upper East Side South → Upper East Side North	61,444	$959,125.48
Upper East Side North → Upper East Side South	53,958	$864,806.57

Route	Trips	Revenue
Upper East Side North → Upper East Side North	42,644	$557,325.90
Upper East Side South → Upper East Side South	39,832	$545,662.68
Midtown Center → Upper East Side South	29,800	$506,857.58
Lincoln Square East → Upper West Side South	25,500	$382,223.95
Upper East Side South → Midtown Center	25,387	$424,638.30
Midtown Center → Upper East Side North	25,164	$548,043.51

Top 5 by revenue — a completely different list:

Route	Trips	Revenue
JFK Airport → Outside of NYC	**16,401**	**$2,065,375.21**
JFK Airport → Times Sq/Theatre District	18,695	$1,760,409.16
LaGuardia Airport → Times Sq/Theatre District	17,197	$1,320,614.80
Upper East Side South → Upper East Side North	61,444	$959,125.48
JFK Airport → Midtown South	10,034	$958,154.89

This is the sharpest finding in the entire analysis. **The busiest route is not remotely the most profitable.** The top route by volume (61,444 trips) earns $959,125. The top route by revenue (16,401 trips — a quarter the volume) earns $2,065,375: **2.2× the revenue from 27% of the trips**, because average revenue per trip is $125.93 versus $15.61.

Four of the five most profitable routes originate at an airport. Only one route appears in both lists.

9.9 Anomaly detection

Run against the **raw** 9,554,778 records, so percentages describe the true original population. A single record can trip several rules, so these categories overlap and must not be summed.

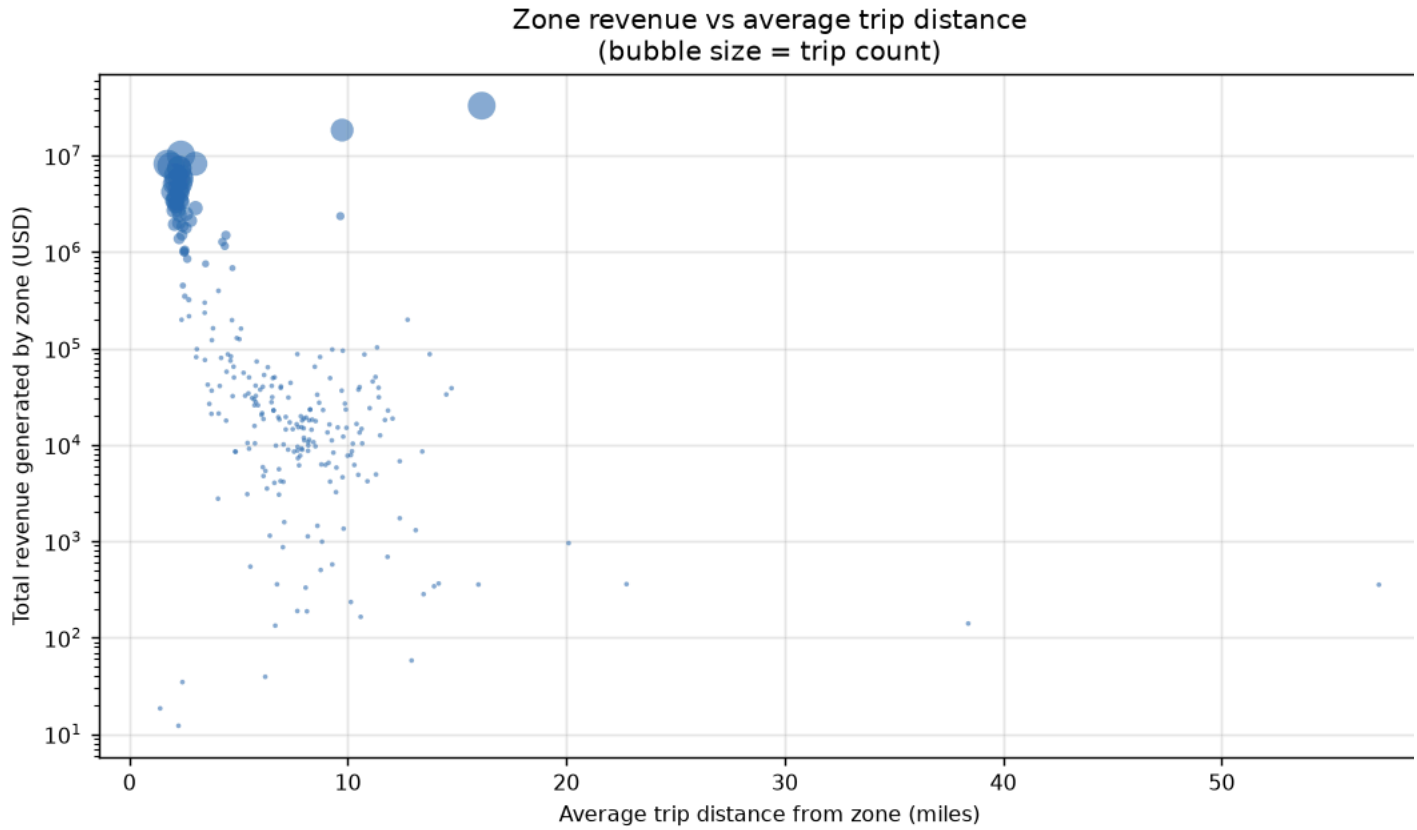
Category	Records	% of raw
Clean (no rule tripped)	**8,460,015**	**88.54%**
Invalid passenger count	857,901	8.98%
Zero or negative distance	215,764	2.26%
Zero or negative fare	139,596	1.46%
Negative total amount	115,895	1.21%
Extreme fare per mile (> $100/mi)	21,507	0.23%
Impossible duration	2,849	0.03%
Extreme distance (> 200 mi)	170	0.002%
Extreme fare (> $1,000)	8	0.0001%

11.46% of the raw dataset trips at least one anomaly rule.

The three largest categories tell three different stories. Invalid passenger counts are a **data-entry** problem. Zero-distance trips (2.26%) are a **metering** problem — the meter engaged and disengaged without recorded movement, consistent with a cancelled hail or GPS failure. Negative fares and totals (1.46% and 1.21%) are almost certainly **accounting reversals** — refunds and adjustments posted through the same pipeline as trips.

The 21,507 records exceeding $100 per mile are the most operationally interesting. They are too numerous to be random noise and are the classic signature of either a meter fault or fare manipulation. At roughly 0.23% of trips they would not distort aggregate statistics, but they are exactly the population a fraud-detection function should be examining case by case.

9.10 Revenue versus distance (Figure 7)



Zone revenue against average trip distance

Each point is a pickup zone, positioned by its average trip distance and total revenue, sized by trip count (revenue on a log scale). The plot separates two distinct business models:

- **High-volume, short-distance Manhattan zones** — large bubbles clustered at 1.5–3 miles, earning revenue through frequency.
- **Lower-volume, long-distance airport zones** — smaller bubbles at 10–16 miles, earning more revenue in total through fare size.

JFK sits alone at the top right: 16.13 miles average distance, $32.9M revenue, on fewer trips than Midtown Center.

10. Multi-Stage MapReduce

10.1 Why a second job is required

Stage 1 (`mapper_revenue.py` → `reducer_revenue.py`) aggregates revenue per pickup zone. It cannot also rank those zones, and the reason is fundamental to the model: **a reducer sees only the values for the keys assigned to it.** The reducer handling zone 132 has no knowledge of zone 138. Ranking is a *global* operation over the complete set of aggregates, so it requires a second pass over stage 1's output.

10.2 Stage 1 output — the intermediate result

Written to `/taxi_project/output/revenue/`, this is a complete standalone dataset (258 zones):

```
001  71     5459.95    628.40    6763.59   76.90  12.38
002  11      503.30    108.79     689.20   45.75  11.82
003  236    9521.60     32.92   10293.98   40.35  10.23
004  9039 145717.21  26400.63  215968.70   16.12   2.73
006  29      544.73    119.47    1124.34   18.78   8.17
...
```

Columns: zone, trips, total fare, total tips, total revenue, average fare, average distance.

10.3 Stage 2 — ranking via the inverted-key technique

Stage 2 reads that HDFS directory as its input. Its central problem is that **MapReduce sorts by key and never by value**, while the ranking we need is by revenue — a value.

The solution is to make revenue the key, with two adjustments:

1. **Invert it**, so that the framework's ascending sort produces descending revenue: `inverted = BIG - round(revenue × 100)`
2. **Zero-pad it** to a fixed width, so lexical string ordering matches numeric ordering: `f"{inverted:015d}"`

Revenue is converted to integer cents first, because floating-point text does not sort correctly as a string. The zone identifier and its metrics travel along in the value.

```
BIG = 10**12
inverted = BIG - int(round(revenue * 100))
print(f"{inverted:015d}\t{zone},{trips},{revenue:.2f},{avg_fare},{avg_dist}")
```

The reducer then does almost nothing — records already arrive in descending revenue order, so it counts to ten and stops. **The framework performed the sort; the reducer merely truncated it.** This is the standard "secondary sort by value" pattern in MapReduce.

`-numReduceTasks 1` is **mandatory** here. With two reducers, each would emit its own top ten from its own partition and neither list would be the global top ten.

10.4 Stage 2 output — Top 10 zones by revenue

Rank	Zone	Trips	Revenue	Avg fare	Avg distance
1	JFK Airport	400,728	$32,945,074.83	$63.74	16.13 mi
2	LaGuardia Airport	274,216	$18,399,346.73	$42.76	9.74 mi
3	Midtown Center	417,601	$10,197,752.86	$15.80	2.36 mi
4	Times Sq/Theatre District	300,937	$8,250,078.88	$18.24	3.02 mi
5	Upper East Side South	409,852	$8,236,010.09	$12.58	1.75 mi
6	Upper East Side North	381,713	$7,839,801.01	$13.02	1.90 mi
7	Midtown East	312,985	$7,442,554.75	$15.36	2.28 mi
8	Penn Station/Madison Sq West	299,896	$7,382,311.26	$16.52	2.29 mi
9	Lincoln Square East	289,311	$6,210,207.58	$13.67	2.12 mi
10	Midtown North	254,334	$6,111,795.04	$15.59	2.40 mi

The ranking **inverts** the trip-count ranking at the top. Midtown Center is first by volume (417,601 trips) but only third by revenue. JFK is third by volume yet first by revenue — earning **3.2× Midtown Center's revenue** on 4% fewer trips, because its average fare is $63.74 against $15.80.

Together the two airports contribute $51.3M — **21.9% of all revenue in the dataset** — from just 8.0% of trips. No analysis based on trip counts alone would have revealed this.

11. YARN Analysis

11.1 Applications executed

All eleven MapReduce jobs ran as YARN applications under `application_1788018839347_*`. Retrieved with `yarn application -list -appStates ALL`:

App ID suffix	Job	Type	Final state	Progress
_0001	clean	MAPREDUCE	SUCCEEDED	100%
_0002	hourly	MAPREDUCE	SUCCEEDED	100%
_0003	daily	MAPREDUCE	SUCCEEDED	100%
_0004	location	MAPREDUCE	SUCCEEDED	100%
_0005	revenue	MAPREDUCE	SUCCEEDED	100%
_0006	payment	MAPREDUCE	SUCCEEDED	100%
_0007	distance	MAPREDUCE	SUCCEEDED	100%
_0008	duration	MAPREDUCE	SUCCEEDED	100%
_0009	route	MAPREDUCE	SUCCEEDED	100%
_0010	anomaly	MAPREDUCE	SUCCEEDED	100%
_0011	topn_revenue	MAPREDUCE	SUCCEEDED	100%

Queue: `root.default`. User: `thierrys`. **Failures: none.**

Full console logs for every job, including complete counter blocks, are in `evidence/`. The ResourceManager UI is at http://localhost:8088/cluster and the NameNode UI at http://localhost:9870.

11.2 What YARN did

YARN separates *resource management* from *job logic*. For each submitted application:

1. The **ResourceManager** accepted the application and allocated a container for its **ApplicationMaster**.
2. The ApplicationMaster requested one container per map task and per reduce task.
3. The **NodeManager** launched each container, localised the shipped `-files` payload (the Python mapper and reducer) into the container's working directory, and executed the task.
4. Task progress was reported back to the ApplicationMaster, which reported to the ResourceManager. On completion the ApplicationMaster exited and its containers were released.

Container localisation is the step that makes `-files` necessary: a container cannot see the submitting machine's filesystem, so any program a task needs must be shipped to it explicitly.

11.3 Splits and parallelism

HDFS stored the 1.02 GB of raw CSV as **9 blocks averaging 118.99 MB** (`hdfs fsck`), with replication 1.0. The cleaning job therefore ran with 9 input splits — the framework created one map task per block and, on a real multi-node cluster, would have scheduled each on the node already holding that block.

On this single-node cluster all containers necessarily run on the same machine, so every map task is trivially "data-local" and the true benefit of locality cannot be demonstrated. This is the central limitation of a pseudo-distributed setup and it is what Section 12 quantifies.

12. Performance Comparison

12.1 Method

The identical analysis — trips, total fare and total revenue grouped by pickup hour, over the same 9,554,778 records with the same validation rules — was implemented twice:

- **Hadoop MapReduce:** `clean` job (raw → cleaned) followed by the `hourly` job.
- **Python/Pandas:** `scripts/pandas_benchmark.py`, reading local CSV in 500,000-row chunks and aggregating per chunk.

12.2 Results

Metric	Python/Pandas	Hadoop MapReduce
Dataset size	1.02 GB (local disk)	1.02 GB (HDFS)
Number of records	9,554,778	9,554,778
Execution time	**8.0 s**	**139 s** (clean 107 s + hourly 32 s)
Peak memory	417 MB	~1.5 GB across JVMs (5 daemons + containers)
Mapper tasks	n/a (1 process)	9 (clean) + 5 (hourly)
Reducer tasks	n/a	4 (clean) + 1 (hourly)
Output size	1.6 KB	1.6 KB + 795 MB intermediate `cleaned/`
Records after cleaning	8,453,548	8,453,547

All 11 MapReduce jobs together took 440 seconds.

12.3 Correctness cross-check

The two implementations agree to within a single record. Hour-by-hour, trip counts and revenue totals are **identical** — for example hour 00: 221,335 trips and $6,307,517.48 in both.

The only differences are at hour 18 (Pandas 609,599 vs Hadoop 609,598) and in the cleaned total (8,453,548 vs 8,453,547). Both differences are **exactly one record**, and both are explained: the MapReduce pipeline removes one duplicate, which the Pandas implementation does not attempt. This is strong evidence that both pipelines are correct.

12.4 Interpretation: Pandas is 17× faster, and that is the expected result

Reporting only the headline would be misleading, so the reasons matter.

Why Hadoop is slower here:

1. **Fixed overhead per job.** Every job pays JVM startup, application submission, container allocation and localisation of the shipped Python files — several seconds before any data is read. Pandas pays none of this.
2. **The shuffle is written to disk.** MapReduce spills mapper output to local disk, sorts it and re-reads it. Pandas aggregates in memory with no serialisation boundary.
3. **Streaming adds a process boundary per task.** Every record is serialised to text, piped to a Python subprocess and parsed back. A native Java MapReduce job would avoid this cost.
4. **One machine cannot parallelise.** MapReduce's advantage is running mappers on many nodes at once. With one node there is nothing to distribute — Hadoop pays the full coordination cost of a distributed system and receives none of its benefit.

Why the comparison nevertheless favours Hadoop as data grows:

The Pandas run succeeded at 417 MB peak memory *because it was written to read in chunks*. A naive `pd.read_csv()` of all 9.5 million rows would have needed several gigabytes. That is the crux: **the Pandas approach has a ceiling set by one machine's RAM, and the MapReduce approach does not.**

Dataset	Pandas	Hadoop MapReduce
1 GB (this project)	8 s — clearly better	139 s — overhead dominates
~50 GB (≈ 4 years of TLC data)	Feasible only with careful chunking; hours	Scales by adding nodes
~500 GB (all TLC vehicle types, full history)	Not feasible on one machine	Routine

The crossover is not a fixed number of records; it is the point at which the dataset stops fitting comfortably on one machine. Below it, distribution is pure overhead. Above it, distribution is the only option that works at all.

The honest conclusion for this dataset: at 1 GB on a single laptop, Pandas is the right tool and Hadoop is 17× slower. The value of the Hadoop implementation is not its speed today but that the *same programs*, unchanged, would run over 500 GB on a 50-node cluster — where the Pandas implementation would not run at all.

13. Business Insights

Direct answers to the twelve questions in Section 15 of the brief.

(a) What is the busiest hour for taxi demand? 18:00 — 609,598 trips, 7.2% of all trips in a single hour, 15.1× the volume of the quietest hour (04:00, 40,315 trips).

(b) What is the busiest day of the week? Thursday — 1,364,036 trips and $38.4M revenue, ahead of Wednesday, Friday and Saturday. Notably not the weekend.

(c) Which pickup zones generate the most trips? Midtown Center (417,601), Upper East Side South (409,852) and JFK Airport (400,728). Eight of the top ten are in Manhattan; the other two are airports.

(d) Which pickup zones generate the most revenue? JFK Airport ($32.9M), LaGuardia Airport ($18.4M) and Midtown Center ($10.2M). The ordering differs from (c): the two airports contribute 21.9% of all revenue from 8.0% of trips.

(e) Which payment method contributes the most revenue? Credit card — $201.6M, or 86.1% of total revenue, across 84.0% of trips.

(f) Do credit-card users generate more tips? They generate more **recorded** tips: 22.54% of fare versus 0.00% for cash. The correct interpretation is that the meter cannot observe cash tips at all. Since average fares are nearly identical ($18.61 card vs $18.71 cash), the two groups take essentially the same trips — the difference is in what the dataset can see, not in customer generosity.

(g) What distance category produces the highest average fare? 20+ miles — $90.30 average fare. But the highest fare *per mile* is the 0–2 mile band at $8.69/mi, versus $3.74/mi for 20+ miles.

(h) What are the most frequently travelled routes? Upper East Side South ↔ Upper East Side North dominates: 61,444 trips one way and 53,958 the other. Eight of the top ten routes are entirely within the Upper East Side and Midtown.

(i) Are the most frequent routes also the most profitable? No — and this is the most commercially significant finding. The busiest route (61,444 trips) earns $959,125. The most profitable route, JFK → Outside of NYC, earns $2,065,375 from 16,401 trips: **2.2× the revenue from 27% of the volume**, at $125.93 per trip versus $15.61. Only one route appears in both top-five lists.

(j) What percentage of records contain potential anomalies? 11.46% of the raw 9,554,778 records trip at least one anomaly rule; 88.54% are clean. The largest category is invalid passenger count (8.98%), which is a data-entry issue rather than an invalid trip. Excluding it, genuinely suspect records fall to roughly 3.5%.

(k) What transportation insights can management derive?

1. **Revenue concentration is not volume concentration.** The two airports produce 21.9% of revenue from 8.0% of trips. Fleet allocation optimised for trip count would systematically under-serve the most valuable segment.
2. **Demand is far flatter across the week than expected** — 1,224,010 trips per weekday versus 1,166,748 per weekend day, a 4.9% gap. Weekend staffing reductions proportional to raw totals would be a serious error.
3. **The evening peak is the binding capacity constraint.** 18:00 carries 15.1× the volume of 04:00; matching supply to that curve is the single largest operational lever.
4. **The market is short-haul.** 58.0% of trips are under 2 miles. Vehicle strategy, driver incentives and pricing should be designed around short urban hops, not long journeys.
5. **Cash-trip earnings are invisible in the data.** Any driver-compensation or tipping analysis built on this dataset understates cash trips by construction.
6. **21,507 trips exceed $100 per mile.** Too numerous to be noise and a classic meter-fault or fare-manipulation signature — a defined population for audit.

(l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing?

Not at this scale. At 1.02 GB on one machine, Pandas completed in 8 seconds against Hadoop's 139 — **17× faster** — because Hadoop pays the full coordination cost of a distributed system (JVM startup, container allocation, disk-based shuffle, one subprocess per task under Streaming) while having only one node to distribute across.

The advantage appears when **the dataset no longer fits on one machine**. The Pandas implementation only succeeded because it was written to read in 500,000-row chunks; its ceiling is set by available RAM. MapReduce has no such ceiling — it scales by adding nodes, and the same mapper and reducer programs used here would run unchanged over 500 GB on a 50-node cluster.

The practical rule: **use the simplest tool that fits the data.** Choose distributed processing when data volume exceeds single-machine memory, when storage must be fault-tolerant across machines, or when the processing window cannot be met by one node — not by default.

14. Limitations

1. Single-node cluster. All five daemons and every container run on one machine, so data locality, network shuffle cost and node-failure recovery cannot be demonstrated. The performance comparison in Section 12 measures Hadoop's overhead without any of its parallelism benefit.

2. The passenger-count rule is probably too strict. Rejecting `passenger_count = 0` discards 851,464 records (8.91%) — the largest single cause of data loss. Those trips almost certainly occurred; the driver simply did not enter an occupancy. Retaining them for the eight analyses that do not use occupancy would raise retention from 88.47% to about 97.4%. The stricter rule was kept for consistency, but it is a defensible point of disagreement.

3. Three months is not a year. January–March 2024 excludes summer tourism, autumn business travel and the December holidays. Seasonal conclusions cannot be drawn, and the winter weather of this window may depress short trips relative to an annual average.

4. Yellow taxis only. Green taxis, for-hire vehicles and high-volume services (Uber, Lyft) are excluded. The Staten Island result — as low as one trip per zone — reflects yellow-cab licensing geography, not actual transport demand there.

5. Cash tips are structurally unobservable. Discussed in Sections 9.5 and 13(f). No analysis of this dataset can measure true tipping behaviour.

6. Zone-level, not coordinate-level. TLC publishes zone IDs rather than latitude/longitude, so within-zone patterns are invisible and "average distance from zone" is a coarse measure.

7. Anomaly thresholds are judgemental. $1,000 maximum fare, 200-mile maximum distance and $100/mile as "extreme" are reasoned but not derived from a statistical model. A distributional approach (percentile- or MAD-based) would be more defensible and is the obvious next step.

8. The 60-minute-plus duration band is unresolved. 116,470 trips averaging 131 minutes at 7.61 mph passed every validity rule but exhibit a profile consistent with meters left running. They are included in the results and flagged rather than removed.

9. CSV conversion inflated storage 6.7×. Necessary for Hadoop Streaming, which cannot read Parquet, but the wrong choice for a production system.

10. `mapper_anomaly.py` **emits one record per rule tripped**, so categories overlap and cannot be summed. Only the CLEAN count is an unambiguous denominator.

15. Conclusion

This project implemented a complete Hadoop-based analytics pipeline over 9,554,778 NYC yellow-taxi trip records — 1.02 GB in HDFS — using eleven MapReduce jobs written as Python programs and executed through Hadoop Streaming. All eleven completed successfully under YARN.

Technically, the work demonstrates the mechanics the assignment set out to teach: HDFS block distribution (9 blocks averaging 119 MB), key-value design under a framework that groups only by key, the Shuffle-and-Sort guarantee that lets every reducer run in O(1) memory, composite keys for multi-field grouping, mapper-side binning to collapse shuffle cardinality, Hadoop counters for auditable data-quality reporting, and the inverted-key technique that makes ranking by value possible in a system that sorts only by key. The two-stage revenue → top-10 workflow shows why some questions genuinely require more than one pass.

Analytically, the recurring theme is that **volume and value are different questions.** Midtown Center leads on trips; JFK leads on revenue by 3.2×. The busiest route earns less than half what the most profitable route earns from four times the trips. The airports produce 21.9% of revenue from 8.0% of trips. Every one of these conclusions is invisible to an analysis that counts trips.

On the technology choice, the honest finding is that Hadoop was the wrong tool for this dataset on this hardware: Pandas completed the same analysis 17× faster. That result is not a failure of the exercise but its most useful lesson. Distributed processing is not free — it costs JVM startup, container allocation, a disk-based shuffle and, under Streaming, a subprocess per task. Those costs buy nothing on a single node. They buy everything once the data outgrows one machine, and the programs written here would run unchanged at that scale.

The correct engineering judgement is therefore not “Hadoop is slow” but **use the simplest tool that fits the data, and know precisely where that tool’s ceiling lies.**

16. References

1. NYC Taxi & Limousine Commission. *TLC Trip Record Data*. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
2. NYC Taxi & Limousine Commission. *Yellow Trips Data Dictionary*. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf
3. NYC Taxi & Limousine Commission. *Taxi Zone Lookup Table*. https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
4. Apache Software Foundation. *Hadoop Streaming Documentation, Release 3.5.0*. https://hadoop.apache.org/docs/r3.5.0/hadoop-streaming/HadoopStreaming.html
5. Apache Software Foundation. *HDFS Architecture Guide*. https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html
6. Apache Software Foundation. *MapReduce Tutorial*. https://hadoop.apache.org/docs/r3.5.0/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html
7. Apache Software Foundation. *Apache Hadoop YARN*. https://hadoop.apache.org/docs/r3.5.0/hadoop-yarn/hadoop-yarn-site/YARN.html
8. Apache Software Foundation. *FileSystem Shell Guide*. https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-common/FileSystemShell.html
9. Dean, J. and Ghemawat, S. (2004). *MapReduce: Simplified Data Processing on Large Clusters*. OSDI’04, 6th Symposium on Operating Systems Design and Implementation.
10. White, T. (2015). *Hadoop: The Definitive Guide*, 4th edition. O’Reilly Media.
11. Apache Software Foundation. *Apache Parquet Documentation*. https://parquet.apache.org/docs/
12. The pandas development team. *pandas documentation*. https://pandas.pydata.org/docs/